Case Study 10: Tuple-Based Route Distance Analysis

Activity sheet case study #50 | Concepts: Tuples, tuple unpacking, operators (exponent), loops, conditional statements, mathematical formula

1. Problem Statement

The stops of a delivery route are stored as tuples containing a label and (x, y) coordinates. Compute the distance between each pair of successive stops using the Euclidean distance formula, identify the longest segment and report total route length and the efficiency of the route compared with a straight line.

2. Objectives

- Store geographical coordinates as tuples and unpack them in loops.
- Apply a given mathematical formula using arithmetic operators and exponents.
- Track the maximum value and its position while iterating.
- Handle special cases: fewer than two points, repeated points and ties.

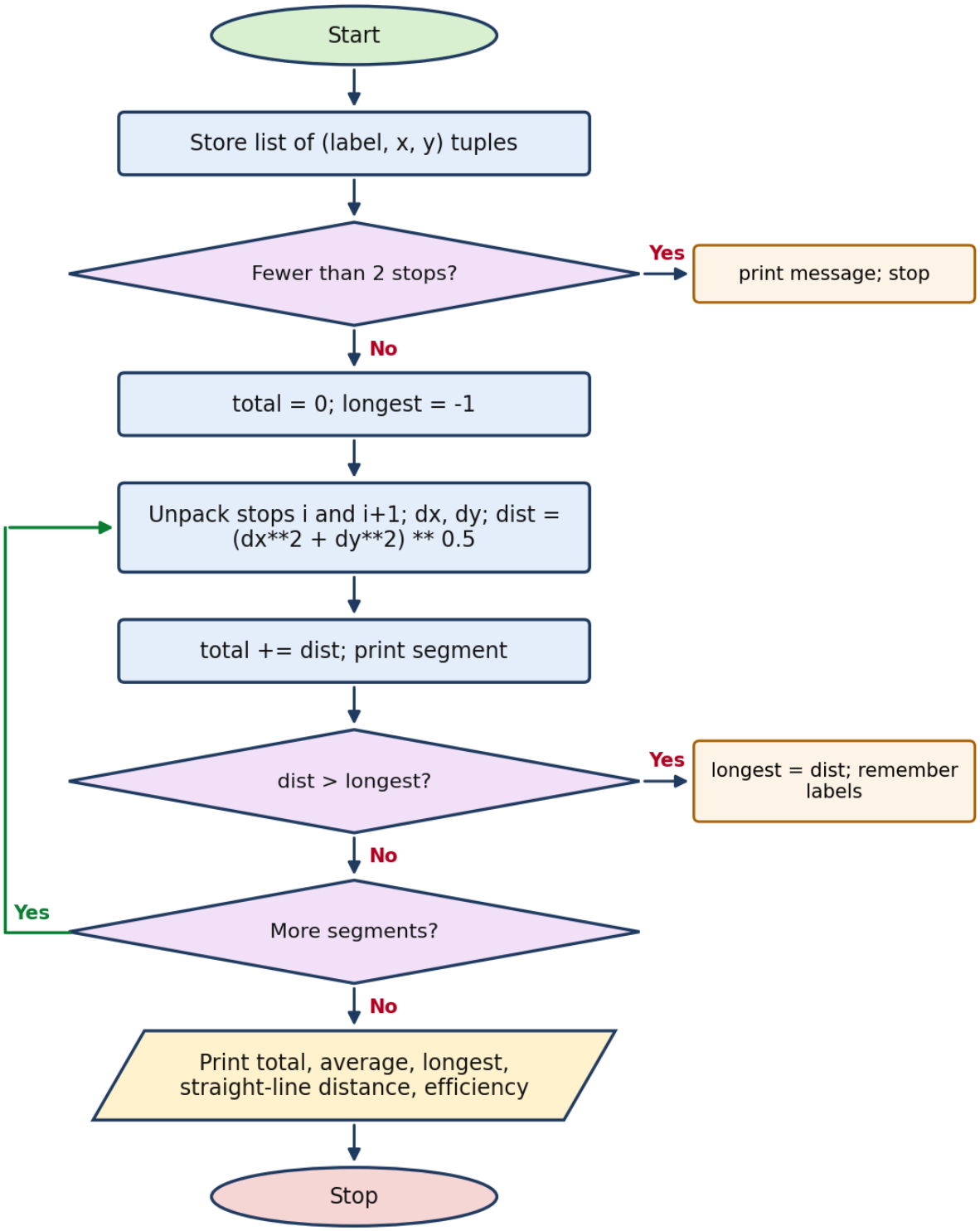
3. Assumptions

- Coordinates are on a flat plane in kilometres; distance = square root of ((x2 - x1)^2 + (y2 - y1)^2), computed as (...) ** 0.5.
- A stop is a tuple (label, x, y); the route visits stops in list order.
- If two segments have the same longest length, the first one is reported.
- Route efficiency = straight-line distance from first to last stop divided by total route length (not defined when the total is zero).

4. Algorithm

1. Store the list of stop tuples.
2. If there are fewer than two stops, print a message and stop.
3. Initialise total = 0, longest = -1.
4. For each index i, unpack stop i and stop i+1 into labels and coordinates.
5. Compute dx, dy and the distance; add it to total and print the segment.
6. If the distance is greater than longest, store it with the two labels.
7. After the loop, print total length, average segment length and the longest segment.
8. Compute the straight-line distance between the first and last stop and, if total > 0, the efficiency percentage.

5. Flowchart



6. Pseudocode

```
START
  READ points (list of tuples: label, x, y)
  IF LENGTH(points) < 2 THEN PRINT "At least two points required" ; STOP
  total <- 0 ; longest <- -1
  FOR i = 0 TO LENGTH(points) - 2
      (name1, x1, y1) <- points[i]
      (name2, x2, y2) <- points[i+1]
      dx <- x2 - x1 ; dy <- y2 - y1
```

```
        dist <- (dx*dx + dy*dy) ^ 0.5
        total <- total + dist
        PRINT name1 -> name2, dist
        IF dist > longest THEN
            longest <- dist ; from_name <- name1 ; to_name <- name2
    END FOR
    PRINT total, total / (number of segments), from_name -> to_name, longest
    straight <- distance between first and last point
    IF total > 0 THEN PRINT straight / total * 100 ELSE PRINT "n/a"
  STOP
```

7. Python Source Code

```
# Case Study 50: Tuple-Based Route Distance Analysis
# ---- INPUT DATA ----
# (label, x, y) in kilometres
points = [("Depot", 0, 0), ("A", 3, 4), ("B", 6, 8),
          ("C", 6, 14), ("D", 2, 11), ("E", 10, 11)]
# ---- END DATA ----

if len(points) < 2:
    print("At least two points are required to form a route.")
else:
    total = 0.0
    longest = -1.0
    longest_from = ""
    longest_to = ""

    print("{:<18}{:>10}".format("Segment", "Distance"))
    print("-" * 28)
    for i in range(len(points) - 1):
        name1, x1, y1 = points[i]            # tuple unpacking
        name2, x2, y2 = points[i + 1]
        dx = x2 - x1
        dy = y2 - y1
        dist = (dx ** 2 + dy ** 2) ** 0.5
        total = total + dist
        print("{:<18}{:>10.2f}".format(name1 + " -> " + name2, dist))
        if dist > longest:
            longest = dist
            longest_from = name1
            longest_to = name2

    segments = len(points) - 1
    print("-" * 28)
    print("Total route length : {:.2f} km".format(total))
    print("Average segment    : {:.2f} km".format(total / segments))
    print("Longest segment    : {} -> {} ({:.2f} km)".format(
          longest_from, longest_to, longest))

    first_name, fx, fy = points[0]
    last_name, lx, ly = points[len(points) - 1]
    straight = ((lx - fx) ** 2 + (ly - fy) ** 2) ** 0.5
    print("Straight-line {} to {}: {:.2f} km".format(first_name, last_name, straight))
    if total > 0:
        print("Route efficiency   : {:.1f}%".format(straight / total * 100))
    else:
        print("Route efficiency   : n/a (total length is zero)")
```

8. Test Cases

#	Test case	Input / data	Expected result	Observed output (actual)	Status
1	Default six-stop route	Depot, A, B, C, D, E	Longest D -> E (8.00 km); total 29.00	`Longest segment : D -> E (8.00 km) | Total route length : 29.00 km | Route efficiency : 51.3%`	**PASS**
2	Single point	one stop	Handled	`At least two points are required to form a route.`	**PASS**
3	Empty list	no stops	Handled	`At least two points are required to form a route.`	**PASS**
4	Negative coordinates	(-3,-4) to (0,0)	3-4-5 triangle gives 5.00	`N -> O 5.00`	**PASS**
5	Tied longest segments	two segments of length 5	First longest reported; straight line gives 100 %	`Longest segment : P -> Q (5.00 km) | Route efficiency : 100.0%`	**PASS**
6	Repeated point (zero length)	same point twice	No division by zero	`Total route length : 0.00 km | Route efficiency : n/a (total length is zero)`	**PASS**
7	Return trip	A to B and back	Efficiency 0 % for a closed loop	`Total route length : 20.00 km | Route efficiency : 0.0%`	**PASS**

9. Output Screenshot

Terminal output of `python case10.py` using the default data:



10. Conclusion

Tuples are a natural fit for fixed (label, x, y) records, and unpacking them in the loop removes index clutter. The program applies the distance formula to every segment, tracks the maximum, and guards against empty routes, ties and zero-length routes.